

软件工程概述

陈龙彪 副教授 博导
厦门大学信息学院
2025年秋学期

学习内容

- 将学习两方面的内容
  - 待开发的产品
  - 为工程技术提供框架的过程
- 主要关注下列问题
  - 到底什么是计算机软件?
  - 为什么要努力建造高质量的基于计算机的系统?
  - 关于软件存在什么样的神话?
  - 什么是软件过程?
  - 是否存在一般性的方法评价一个过程的质量?
  - 软件开发可以应用什么过程模型，它们的优点和缺点?
  - 在软件工程中已提出了什么高级的过程模型?

第1章：产品

预言

- 计算机软件已经成为一种驱动力
  - 它是进行商业决策的引擎
  - 它是现代科学研究和工程问题寻求解答的基础
  - 它也是鉴别现代产品和服务的关键因素
- 它被嵌入在各种类型的系统中
  - 交通、医疗、电信、军事、工业生产过程、娱乐、办公……
  - 软件在现代社会中确实是必不可少的
- 进入21世纪，软件将成为从基础教育到基因工程的所有领域新进展的驱动器



但是...


人类战胜"千年虫" 顺利进入2000年
美洲
亚洲
欧洲
非洲
北京


A problem has been detected and windows has been shut down to prevent damage to your computer.


Y2K
01/01/00
创世纪
GENESIS
千年虫（公元1999）


BIENVENUE A
L'ECOLE CENTRALE
DE NANTES
12 HEURES 09
3 JANVIER 1900

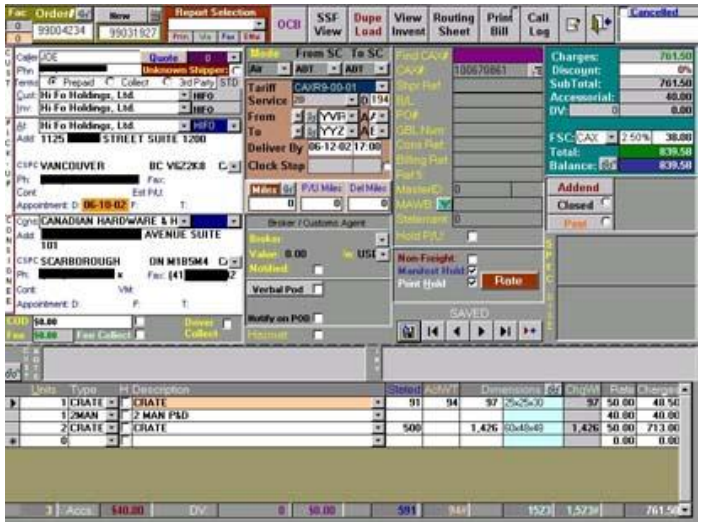

WORLD

软件越来越复杂

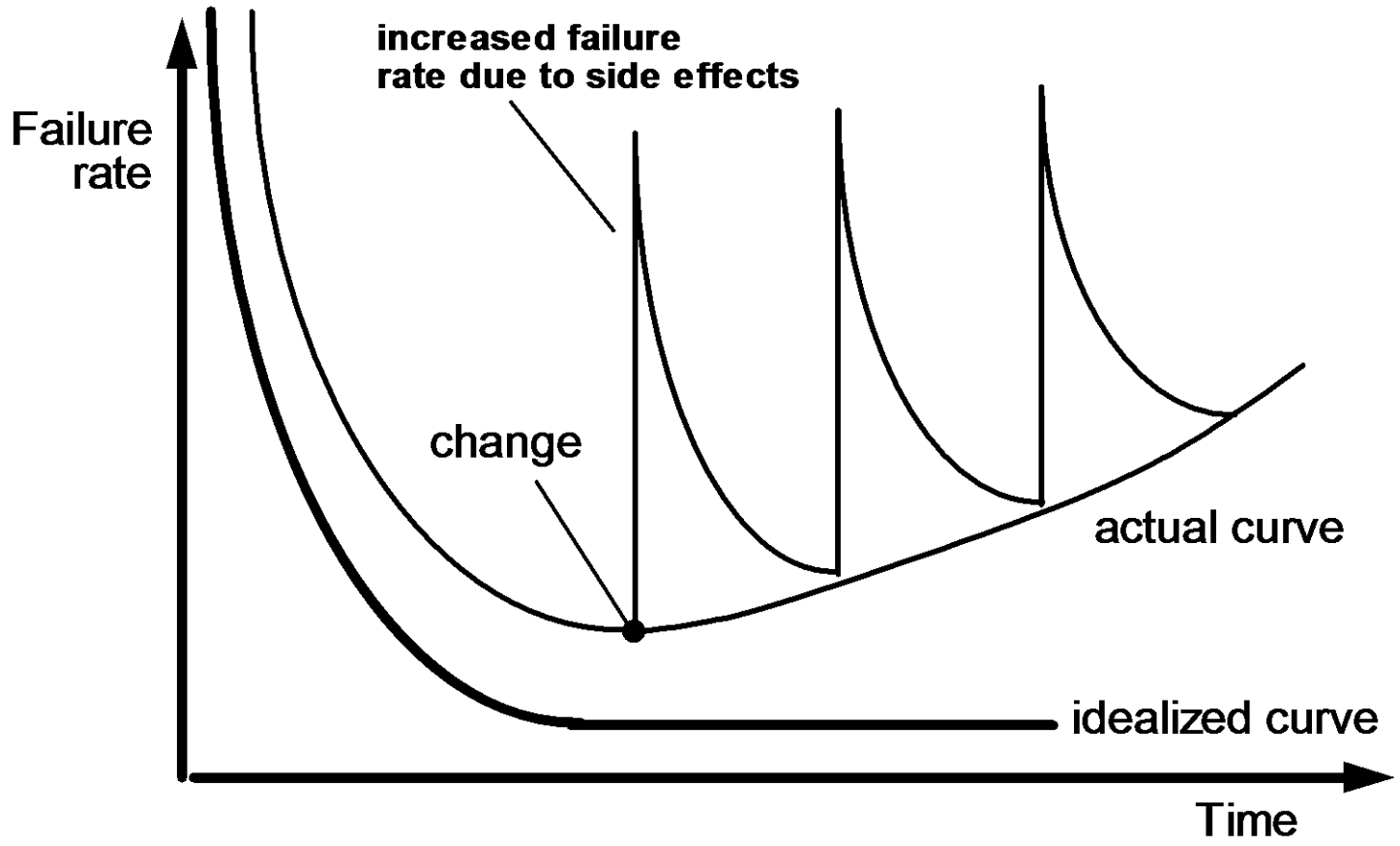
- BMW 7系列轿车
  - 67个控制单元
  - 4套汽车系统
  - 约270个软件功能
  - 软件规模已达65MB

新车中90%
的
投资用于软件



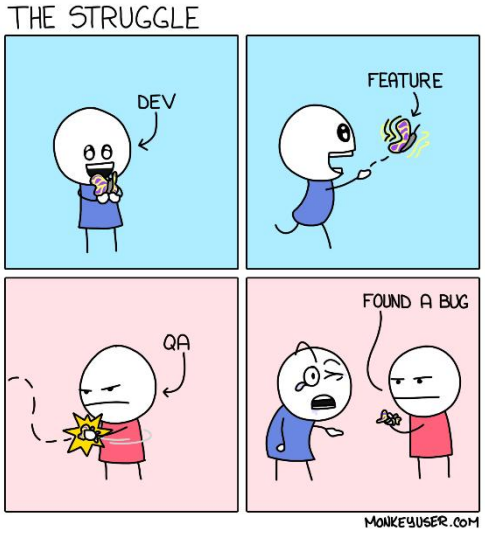


软件的理想故障曲线和实际故障曲线



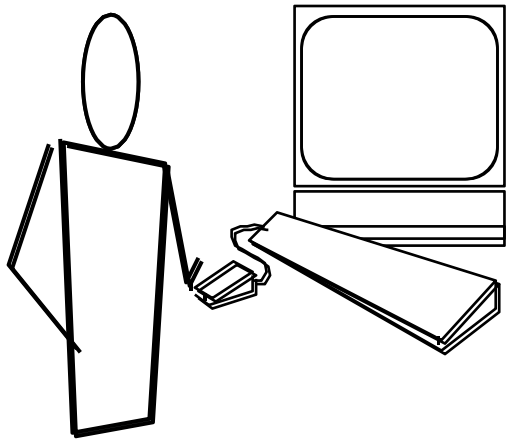
软件开发中的常见问题

- 为什么需要那么长时间才能结束开发?
- 为什么成本如此之高?
- 为什么不能在把软件交付给客户之前就发现所有的错误?
- 为什么在软件开发过程中我们总是难以度量其进展?



软件是什么

- 程序：它被执行时提供希望功能和性能的指令
  - 程序是按事先设计的功能和性能要求执行的指令序列
- 数据：使得程序能够适当地操作信息的数据结构
  - 如文字、图形、图像、声音等
- 文档：描述程序操作和使用的文档
  - 与程序开发、维护和使用有关的阐明性资料，文档一般是给人看的，不一定装入计算机



软件角色的演化

- 软件的双重角色
  - 软件系统一个产品
    - 表达了由计算机硬件或更广地说可通过局部硬件访问的计算机网络所体现的计算潜能
    - 是一个信息变换器——产生、管理、获取、修改、显示或传送信 息，这些信息可以简单或复杂
  - 同时又是产品交付使用的载体
    - 是计算机的控制（操作系统）的基础、信息通信（网络）的基础 和其他程序的创建和控制的基础（软件工具和环境）
- 软件提供了我们这个时代最重要的产品——信息

软件的特征

- 软件是逻辑的而不是有形的系统元件
- 软件具有与硬件完全不同的特征
  - 软件是被开发或设计的，而不是传统意义上被制造的
  - 软件不会“磨损”，但软件维护要比硬件维护复杂得多
  - 虽然软件正在向基于构件的组装前进，大多数软件仍然是定制的



软件的其他特点

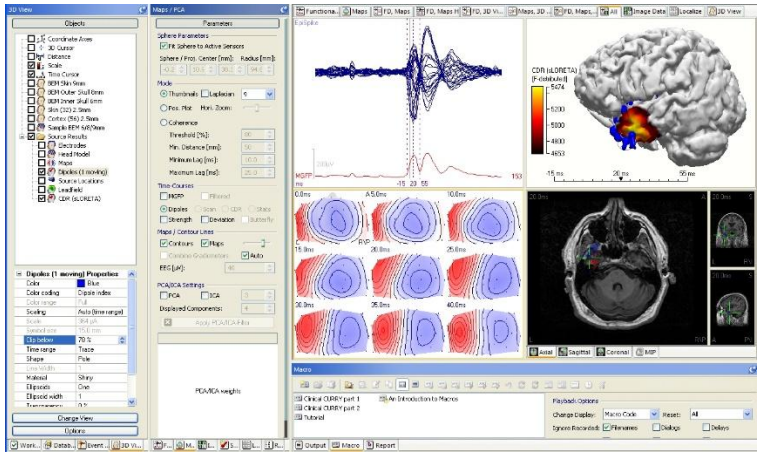
- 软件的开发和运行常受到计算机硬件的限制，对计算机硬件有着不同程度的依赖性
- 软件的开发至今尚未完全实现自动化
- 软件成本相当昂贵
- 相当多的软件工作涉及到社会因素

软件的分类

- 系统软件
  - 属于计算机系统中最靠近硬件的一层，其它软件一般都通过系统软件发挥作用，它与具体的应用领域无关
    - 如操作系统、编译程序等
- 支持软件
  - 支持软件的开发和维护的软件
    - 如数据库管理系统、网络软件、软件开发环境等
- 应用软件
  - 特定应用领域专用的软件
    - 如实时软件、嵌入式软件、科学和工程计算软件、事务处理软件、人工智能软件等

软件的分类

- 按软件工作方式划分：
  - 实时处理软件
  - 分时软件
  - 交互式软件
  - 批处理软件
- 按软件服务对象的范围划分：
  - 项目软件
  - 产品软件
- 按软件失效的影响进行划分：
  - 高可靠性软件
  - 一般可靠性软件





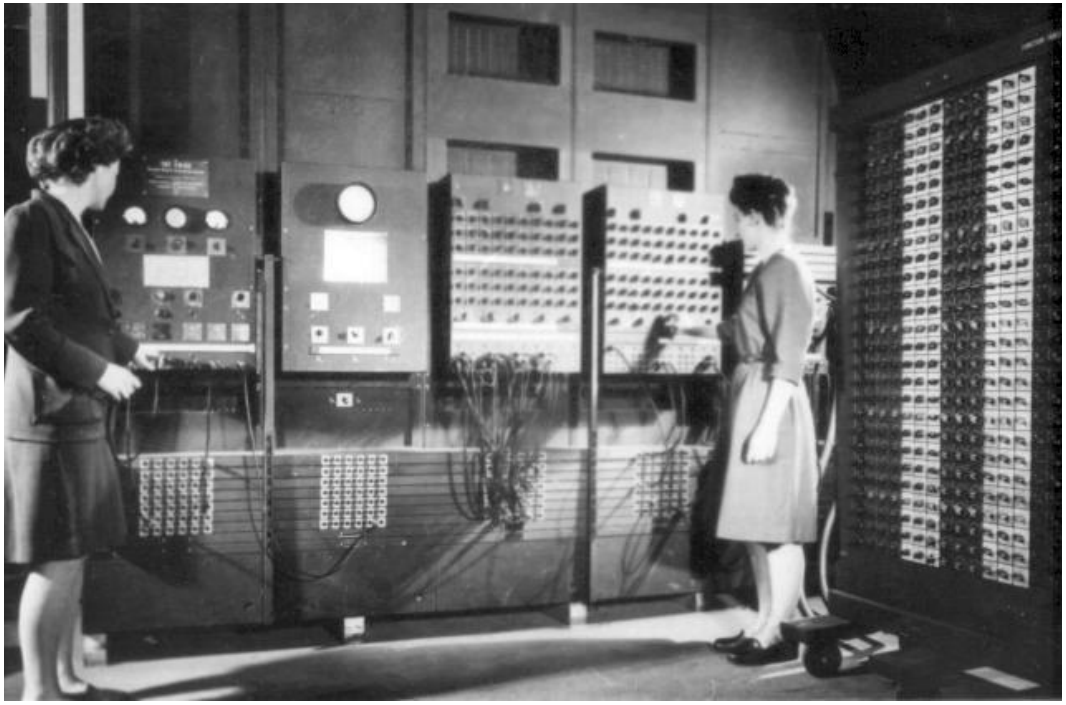
软件的发展进程

- 1946 – 1956 年：从计算机问世到实用高级程序语言出现前
  - 存储容量比较小，运算速度比较慢
  - 采用个体工作方式，用低级语言编写程序
  - 应用领域主要是以数值数据处理为主的科学计算，其特点是输入、 输出量较小
  - 衡量程序质量的标准主要是功效，即运行时间省、占用内存小
  - 主要研究内容是科学计算程序、服务性程序和程序库
  - 主要研究对象是顺序程序



ENIAC

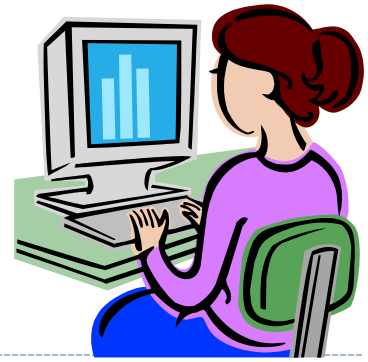
- 1946年2月14日，世界上第一台电脑ENIAC在美国宾夕法尼亚大学诞生



Programmers Betty J. Jennings (left) and Fran Bilas (right) operate the ENIAC

软件的发展进程

- **1956－1968 年：实用高级程序语言出现到软件工程出现前**
  - 存储器容量大，外围设备得到迅速发展，出现了高级程序设计语言
  - 应用领域包括数据处理（非数值数据），其特点是计算量不大，但输入、输出量却较大
  - 高速主机与低速外围设备的矛盾突出，出现了操作系统、并发程序、数据库及其管理系统
  - 20世纪60年代初提出了软件一词，开始认识到文档的重要性
  - 研究高级程序设计语言、编译程序、操作系统、支持编程的工具及各种应用软件
  - 工作方式逐步从个体方式转向合作方式
  - 出现软件危机



软件的发展进程

- **1968年 – 至今：从软件工程出现到现在**
  - 硬件向巨型机和微型机二个方向发展，出现了计算机网络，软件方面提出了软件工程，出现了“计算机辅助软件工程”（CASE）
  - 计算机的应用领域渗透到各个业务领域，出现了嵌入式应用，其特点是受制于它所嵌入的宿主系统
  - 开发方式逐步由个体合作方式转向工程方式
  - 软件工程方面的研究主要包括软件开发模型、软件开发方法及技术、软件工具与环境、软件过程、软件自动化系统等
  - 软件方面研究以智能化、自动化、集成化、并行化、以及自然化为标志的软件开发新技术



软件的发展进程

- **1968年 – 至今：从软件工程出现到现在**

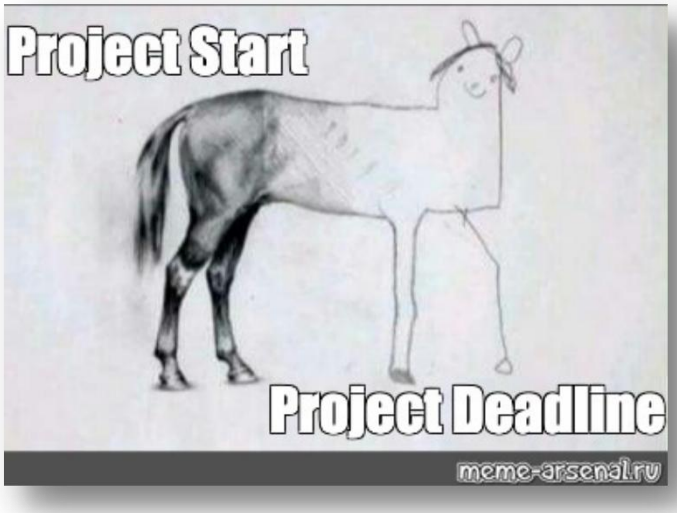


软件危机

许多软件项目不能满足客户的要求

许多软件项目超出预算和时间安排



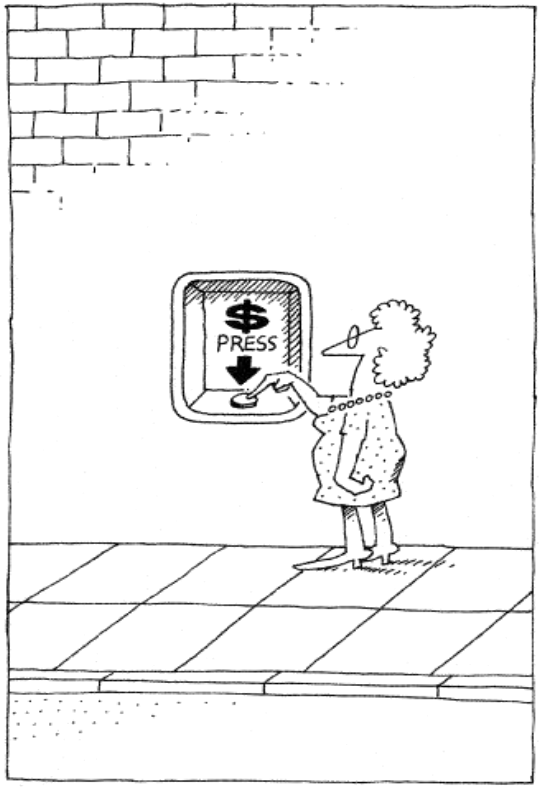


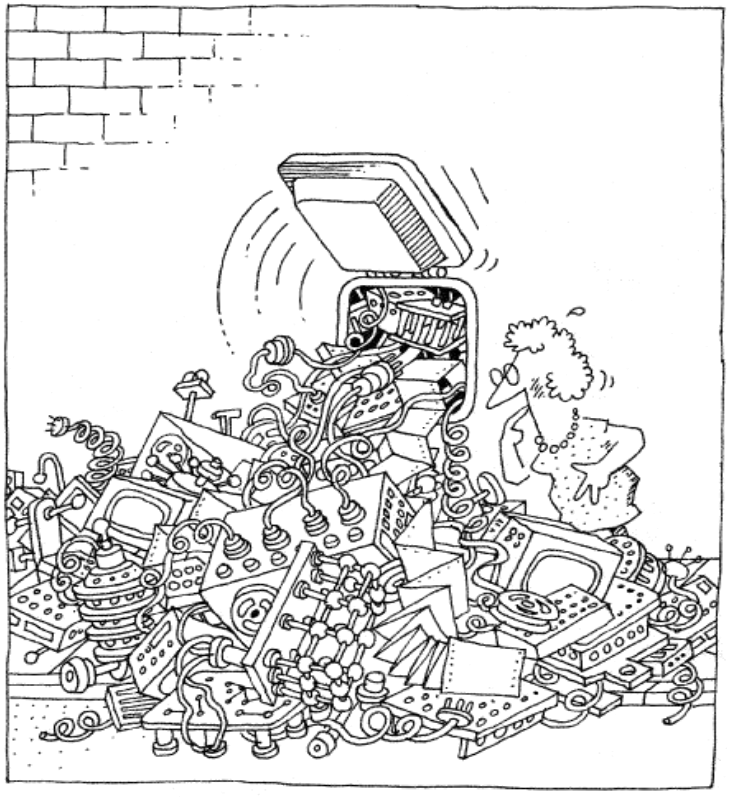
软件危机的表现

- 对软件开发成本和进度的估计常常很不正确
- 用户对“已完成的”软件系统不满意的现象经常发生
- 软件产品的质量往往靠不住
- 软件常常是不可维护的
- 软件通常没有适当的文档资料
- 软件成本在计算机系统总成本中所占的比例逐年上升
- 软件开发生产率提高的速度远远跟不上计算机应用迅速普及深入的趋势

软件危机的表现

- 对软件开发成本和进度的估计常常很不正确
- 用户对"已完成的"软件系统不满意的现象经常发生
- 软件
- 软件
- 软件
- 软件
- 软件





速普及深

软件危机的原因

- 软件是逻辑产品，开发进度、成本难以估计
- 缺乏或不完整、不一致的文档给维护带来困难
- 用户对软件需求的[illegible]有遗漏，有二义性
- 软件开发人员对需[illegible]来愿望有差异

请帮我设计一个
五颜六色的黑



- 大型软件项目需多[illegible]理经验
- 开发人员不能有效[illegible]软件的全部关系
- 缺乏有力的方法学[illegible]
- 软件项目的特殊性[illegible]

软件危机的原因

- 软件是逻辑产品，估计
- 缺乏或不完整、不来困难
- 用户对软件需求的有遗漏，有二义性
- 软件开发人员对需来愿望有差异



- 大型软件项目需多理经验
- 开发人员不能有效软件的全部关系
- 缺乏有力的方法学
- 软件项目的特殊性

克服软件危机的途径

- 消除错误的概念和做法
- 推广使用成功的开发技术和方法
- 使用软件工具和软件工程支持环境





软件神话

- 管理神话
  - 神话：如果已落后于计划，可增加更多的程序员来赶上
  - 现实：原来正在工作的开发者必须花时间来培训新人，这样就减少了他们花在项目开发上的时间。人手可以增加，但只能是在计划稠密、协调良好的情况下
- 客户神话
  - 神话：项目需求总是在不断变更，但这些变更能够很容易满足，因为软件是灵活的
  - 现实：软件需求确实是经常变更的，但这些变更产生的影响会随着其被引入的时间而不同

软件神话

- 管理神话
  - 神话：如果已落后于计划，可增加更多的程序员来赶上
  - 现实：原来正在工作的开发者必须花时间来培训新人，这样就减少了他们花在项目开发上的时间。人手可以增加，但只能是在计划稠密、协调良好的情况下
- 客户神话
  - 神话：项目需求总是在不断变更，但这些变更能够很容易满足，因为软件是灵活的
  - 现实：软件需求确实是经常变更的，但这些变更产生的影响会随着其被引入的时间而不同

第2章：过程

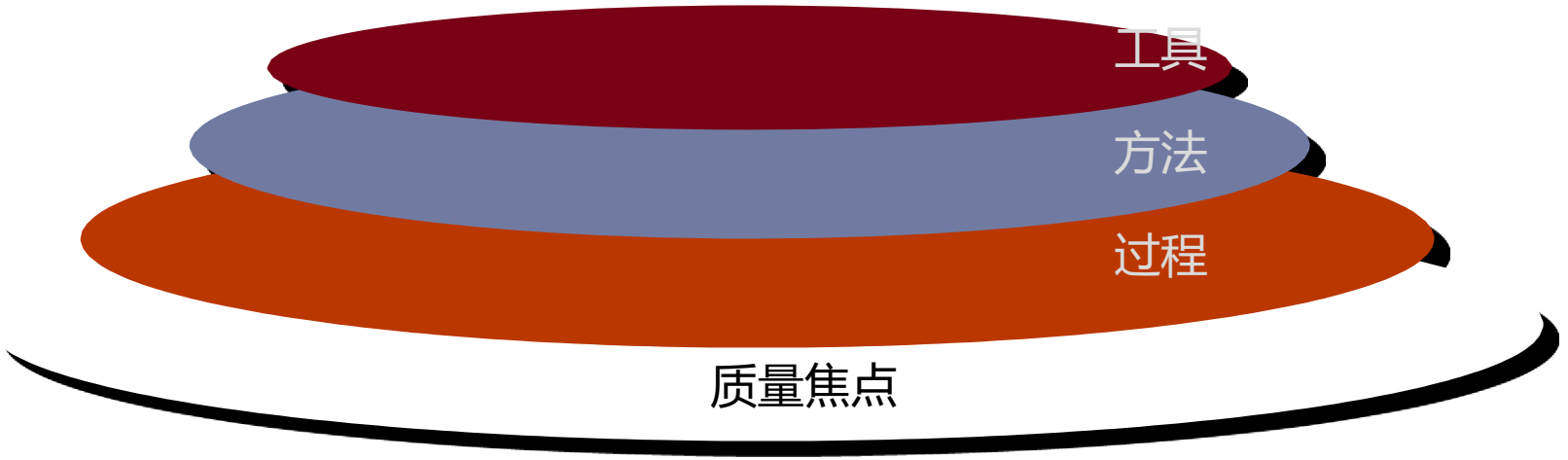


软件过程

- 软件过程为一个为建造高质量软件所需完成的任务的框架
  - 一个软件过程定义了软件开发中采用的方法
  - 软件工程还包含该过程中应用的技术
- 软件工程是由有创造力、有知识的人在定义好的、成熟的软件过程中进行的，该过程适合于他们建造的产品和他们的市场需求

软件工程：一种层次化技术

- Software engineering is an engineering discipline that is concerned with all aspects of software production
  - 支持软件工程的根基在于对质量的关注
  - 软件工程的基础是过程层
  - 方法：需求分析、设计、编程、测试和支持
  - 工具：对过程和方法提供了自动的或半自动的支持



软件工程定义

- Fritz Bauer：软件工程是为了经济地获得可靠的和能在实际机器上高效运行的软件而建立和使用的好的工程原则

- IEEE：软件工程是（1）将系统化的、规范的、可度量的方法应用于软件的开发、运行和维护的过程，即将工程化应用于软件中；（2）在（1）中所述方法的研究

- 计算机科学技术百科全书：软件工程是应用计算机科学、 数学及管理科学等原理，以工程化的原则和方法制作软件的工程

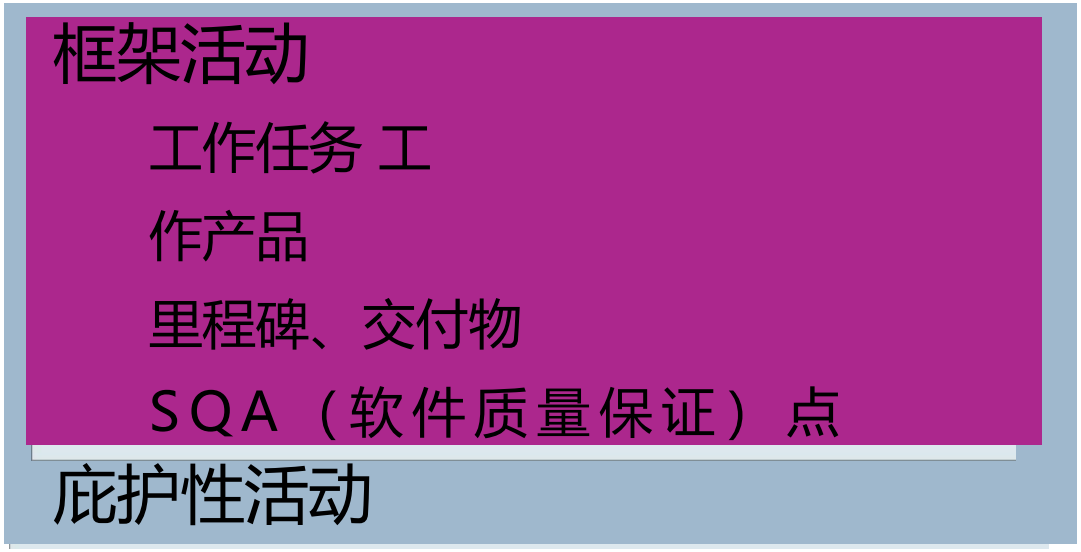
软件工程的一般视图

- 定义阶段集中于“做什么”
  - 系统或信息工程、软件项目计划和需求分析
- 开发阶段集中于“如何做”
  - 软件设计、代码生成和软件测试
- 支持阶段关注于“变化”
  - 纠错、适应、增强、预防

软件过程

- 生产一个最终满足需求且达到工程目标的软件产品所需要的步骤
- 软件工程过程包括：开发过程、运作过程、维护过程、管理过程、支持过程、获取过程、供应过程、剪裁过程等

公共过程框架



能力成熟度模型（CMMI）

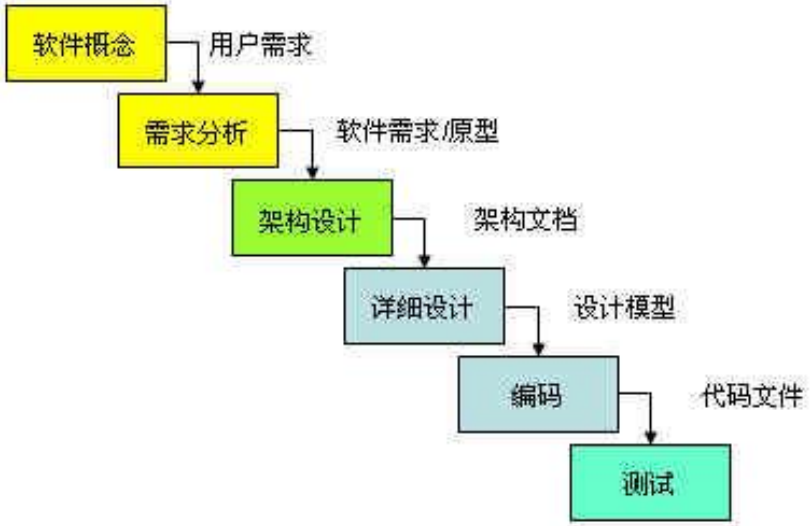
- 在不同的过程成熟度级别上所需要的关键活动
  - 第一级：初始级
    - 几乎没有过程定义，成功完全取决于个人的能力
  - 第二级：可重复级
    - 有适当的必要的过程规范，可以重复与以前类似的项目的成功
  - 第三级：定义级
    - 用于管理和工程活动的软件过程已经文档化、标准化并与整个组织的软件过程相集成
  - 第四级：管理级
    - 软件过程和产品质量的详细度量数据被收集
  - 第五级：优化级
    - 通过定量反馈进行不断的过程改进

关键过程区域（KPA）

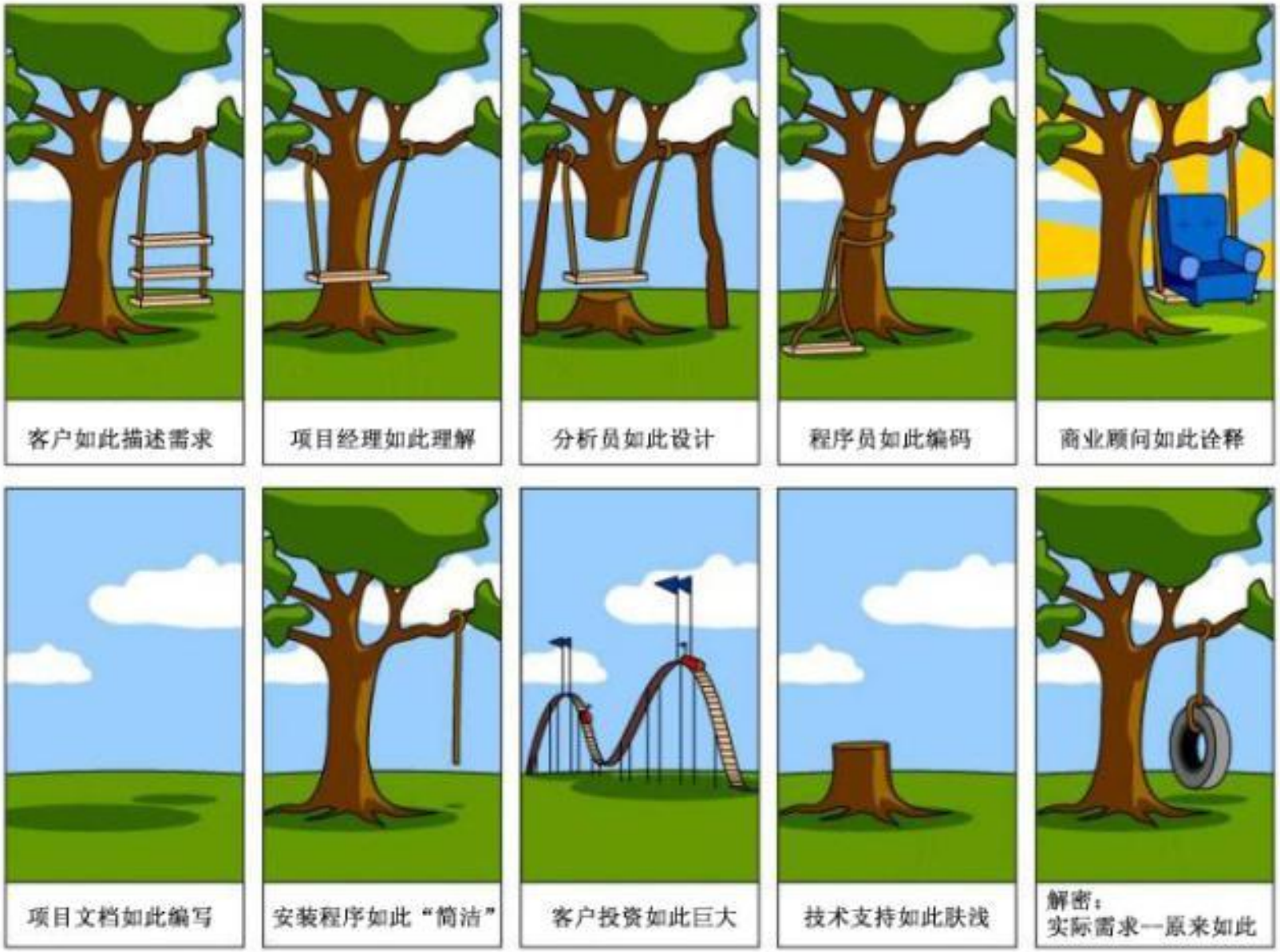
- 将关键过程区域与每一个成熟度级别联系起来
- 每一个KPA均通过标识下列特征来描述
  - 目标——KPA必须达到的总体目标
  - 承诺——组织必须遵守的需求
  - 能力——必须存在以使组织能够满足承诺的那些事务
  - 活动——为了完成KPA的功能所需要的特定任务
  - 监控实现的方法——活动在实现过程中被监控的方式
  - 验证实现的方法——KPA的适当的实践能被验证的方式

软件生命周期（SDLC）

- 软件有一个孕育、诞生、成长、成熟、衰亡的生存过程。这个过程即为计算机软件的生存周期
- 软件生存周期大体可分为如下几个活动：计算机系统工程、需求分析、设计、编码、测试、运行和维护

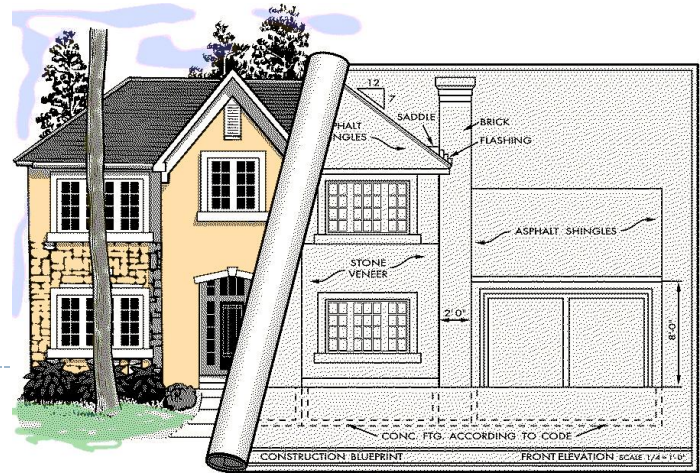


软件生命周期（SDLC）


客户如此描述需求
项目经理如此理解
分析员如此设计
程序员如此编码
商业顾问如此诠释
项目文档如此编写
安装程序如此“简洁”
客户投资如此巨大
技术支持如此肤浅
解密：
实际需求一原来如此

系统工程

- 计算机系统包括计算机硬件、软件、使用计算机系统的人、数据库、文档、规程等系统元素
- 计算机系统工程的任务
  - 确定待开发软件的总体要求和范围，以及它与其它计算机系统元素之间的关系
  - 进行成本估算，做出进度安排
  - 进行可行性分析，即从经济、技术、法律等方面分析待开发的软件是否有可行的解决方案，并在若干个可行的解决方案中作出选择

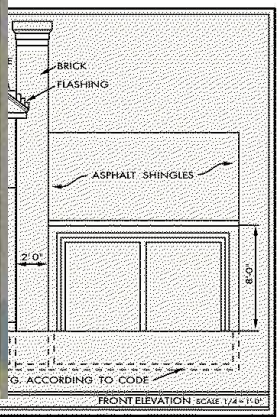


系统

- 计算
  系统
- 计算
  - 确
    机
  - 进
  - 进
    开
    解







软件需求分析

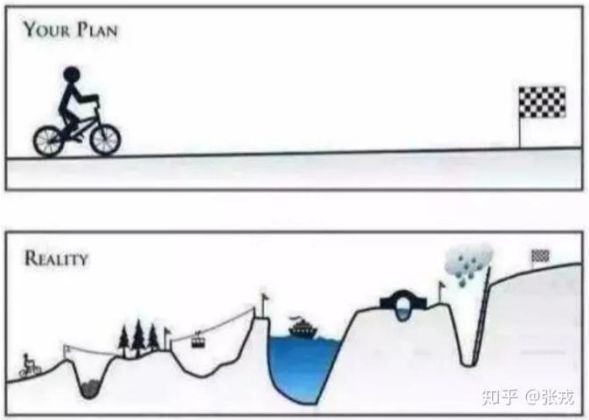
- 主要解决待开发软件要“做什么”的问题
- 确定软件的功能、性能、数据、界面等要求，生成软件需求规约

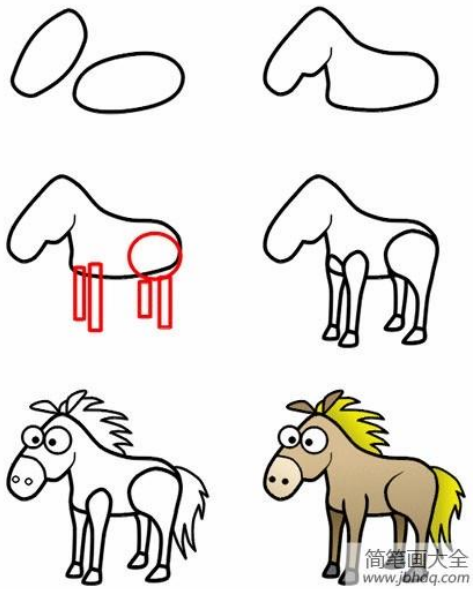
经典的“四拍”

- ✓ **决策时拍脑袋——就这么定了**
- ✓ **指挥时拍胸脯——保证没问题**
- ✓ **失误时拍大腿——我怎么木想到**
- ✓ **追查时拍屁股——老子不干了**

软件设计

- 主要解决待开发软件“怎么做”的问题
- 软件设计通常可分为总体设计和详细设计
  - 总体设计的任务是设计软件系统的体系结构，包括软件系统的组成成分、各成分的功能和接口、 成分间的连接和通信，同时设计全局数据结构
  - 详细设计的任务是设计各个组成成分的实现细节， 包括局部数据结构和算法等





编码、测试、 维护

- 编码
  - 用程序设计语言将设计的结果转换为可执行的程序代码
- 测试
  - 发现并纠正软件中的错误和缺陷。测试主要包括单元测试、集成测试、确认测试和系统测试
- 运行和维护
  - 在软件运行期间，当发现了软件中潜藏的错误或需要增加新的功能或使软件适应外界环境的变化等情况出现时，对软件进行修改



一个测试工程师走进一家酒吧，要了一杯啤酒

一个测试工程师走进一家酒吧，要了一杯咖啡

一个测试工程师走进一家酒吧，要了0.7杯啤酒

一个测试工程师走进一家酒吧，要了-1杯啤酒

一个测试工程师走进一家酒吧，要了2^32杯啤酒

一个测试工程师走进一家酒吧，要了一杯洗脚水

一个测试工程师走进一家酒吧，要了一杯蜥蜴

一个测试工程师走进一家酒吧，要了一份asdfQwer@24dg!&*(@

一个测试工程师走进一家酒吧，什么也没要

一个测试工程师走进一家酒吧，又走出去又从窗户进来又从后门出去从下水道钻进来

一个测试工程师走进一家酒吧，又走出去又进来又出去又进来又出去，最后在外面把老板打了一顿

一个测试工程师走进一

一个测试工程师走进一家酒吧，要了一杯烫烫烫的锟斤拷

一个测试工程师走进一家酒吧，要了NaN杯Null

1T测试工程师冲进一家酒吧，要了500T啤酒咖啡洗脚水野猫狼牙棒奶茶

1T测试工程师把酒吧拆了

一个测试工程师化装成老板走进一家酒吧，要了500杯啤酒并且不付钱

一万个测试工程师在酒吧门外呼啸而过

一个测试工程师走进一家酒吧，要了一杯啤酒';DROP TABLE 酒吧

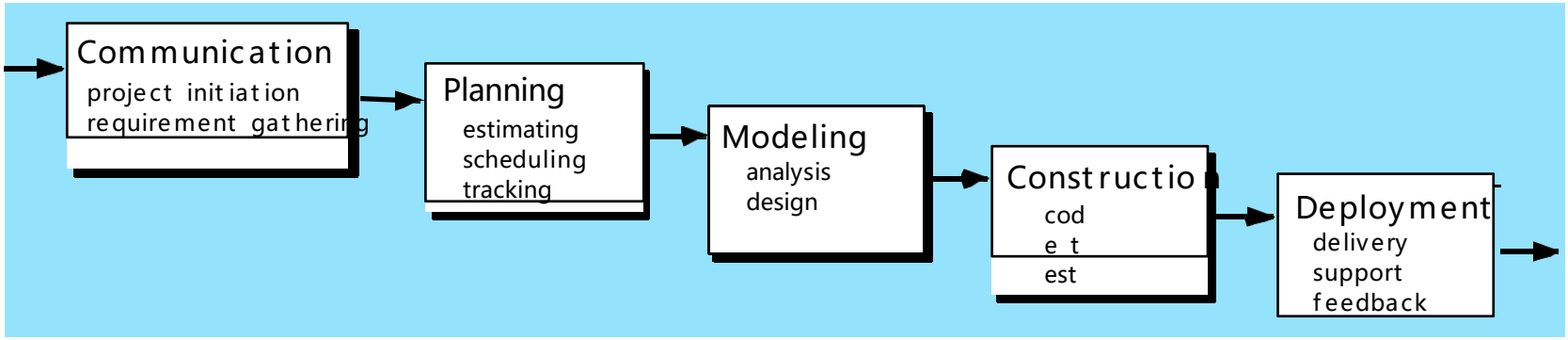
测试工程师们满意地离开了酒吧。

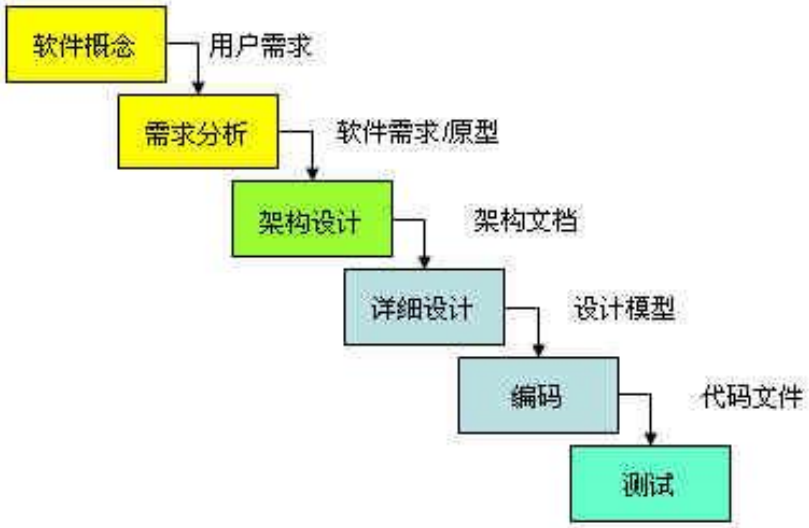
然后一名顾客点了一份炒饭，酒吧炸了

软件过程模型

- 软件开发全部过程、活动和任务的结构框架
- 典型的软件过程模型
  - 瀑布模型（waterfall model）
  - 演化模型（evolutionary model）
  - 增量模型（incremental model）
  - 原型模型（prototyping model）
  - 螺旋模型（spiral model）
  - 喷泉模型（water fountain model）
  - 基于构件的开发模型（component-based development model）
  - 形式方法模型（formal methods model）

瀑布模型


Communication
project initiation
requirement gathering
Planning
estimating
scheduling
tracking
Modeling
analysis
design
Construction
code
test
Deployment
delivery
support
feedback


软件概念
用户需求
需求分析
软件需求/原型
架构设计
架构文档
详细设计
设计模型
编码
代码文件
测试

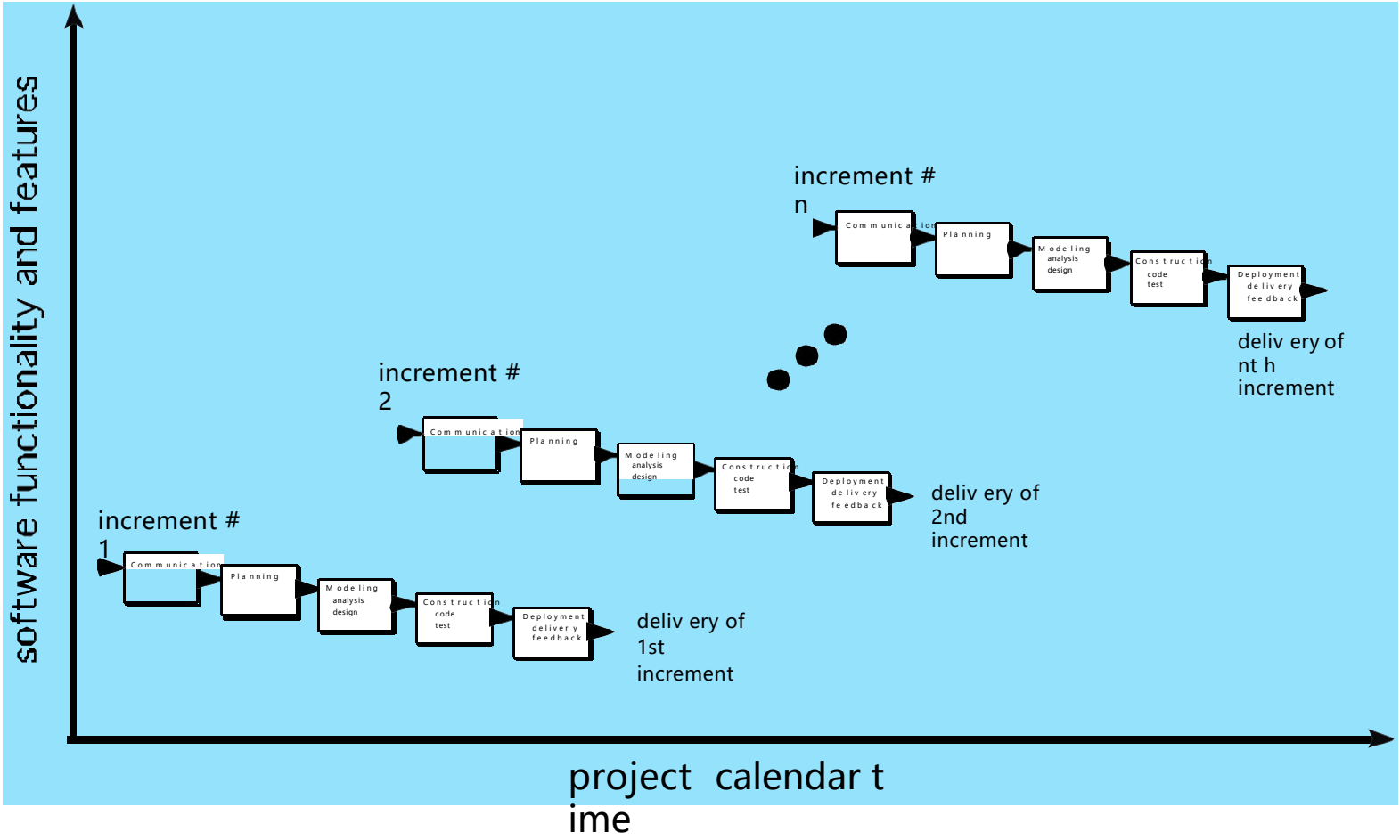
瀑布模型

- 1970年W. Royce提出瀑布模型
- 特征
  - 接受上一阶段的结果作为本阶段的输入
  - 利用这一输入实施本阶段应完成的活动
  - 对本阶段的工作进行评审
  - 将本阶段的结果作为输出，传递给下一阶段
- 缺点
  - 缺乏灵活性，难以适应需求不明确或需求经常变化的软件开发
  - 开发早期存在的问题往往要到交付使用时才发现，维护代价大

演化模型

- 许多软件项目在开发早期对软件需求的认识是模糊的、不确定的，因此软件很难一次开发成功
- 可以在获取了一组基本的需求后，通过快速分析构造出该软件的一个初始可运行版本，称之谓原型（prototype），然后根据用户在试用原型的过程中提 出的意见和建议、或者增加新的需求，对原型进行改 造，获得原型的新版本，重复这一过程，最终得到令 客户满意的软件产品
- 演化模型的开发过程就是从构造初始的原型出发，逐步将其演化成最终软件产品的过程
- 演化模型适用于对软件需求缺乏准确认识的情况
- 典型的演化模型有：增量模型、原型模型、螺旋模型

增量模型


software functionality and features
increment # 1
Communication
Planning
Modeling analysis design
Construction code test
Deployment delivery feedback
deliv ery of 1st increment
increment # 2
Communication
Planning
Modeling analysis design
Construction code test
Deployment delivery feedback
deliv ery of 2nd increment
increment # n
Communication
Planning
Modeling analysis design
Construction code test
Deployment delivery feedback
deliv ery of nt h increment
project calendar t ime

增量模型

- 增量模型将软件的开发过程分成若干个日程时间 交错的线性序列，每个线性序列产生软件的一个可发布的“增量”版本，后一个版本是对前一版 本的修改和补充，重复增量发布的过程，直至产生最终的完善产品
- 增量模型融合了瀑布模型的基本成分（重复地应用）和演化模型的迭代特征
- 增量模型强调每一个增量都发布一个可运行的产品





增量模型

- 增量模型特别适用于
  - 需求经常变化的软件开发
  - 市场急需而开发人员和资金不能在设定的市场期限之前实现一个完善的产品的软件开发
- 增量模型能有计划地管理技术风险，如早期增量版本中避免采用尚未成熟的技术



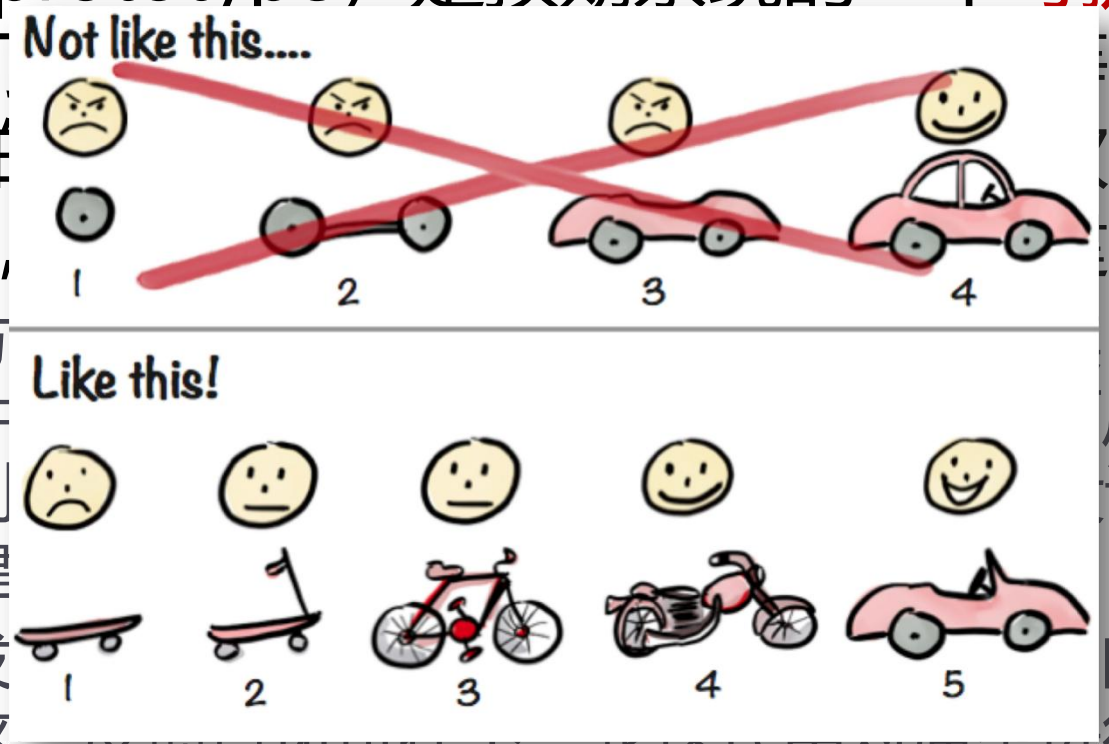


原型模型

- 原型（prototype）是预期系统的一个可执行版本，它反映了系统性质（如功能、计算结果等）的一个选定的子集。一个原型不必满足目标软件的所有约束，其目的是能快速、低成本地构建原型
  - 原型方法从软件工程师与客户的交流开始，其目的是定义软件的总体目标，标识需求。然后快速制订原型开发的计划，确定原型的目标和范围，采用快速设计的方式对其建模，并构建原型
  - 被开发的原型应交付给客户试用，并收集客户的反馈意见，这些反馈意见可在下一轮迭代中对原型进行改进。在前一个原型需要改进，或者需要扩展其范围的时候，进入下一轮原型的迭代开发

原型模型

- 原型（prototype）是预期系统的一个可执行版本，它反映[illegible]）的一个选定[illegible]件的所有约束，[illegible]原型
  - 原型方[illegible]目的是定义软件[illegible]原型开发的计划[illegible]计的方式对其建[illegible]
  - 被开发[illegible]的反馈意见，这[illegible]行改进。在前一个原型需要改进，或者需要扩展其范围的时候，进入下一轮原型的迭代开发



原型模型



太阳能充电宝



新能源汽车

原型模型

- 原型的类型
  - 探索型（exploratory prototyping）
    - 其目的是要弄清目标系统的要求，确定所希望的特性，并探讨多种方案的可行性
  - 实验型（experimental prototyping）
    - 其目的是验证方案或算法的合理性，它是在大规模开发和实现前，用于考核方案是否合适，规格说明是否可靠
  - 演化型（evolutionary prototyping）
    - 其目的是将原型作为目标系统的一部分，通过对原型的多次改进，逐步将原型演化成最终的目标系统

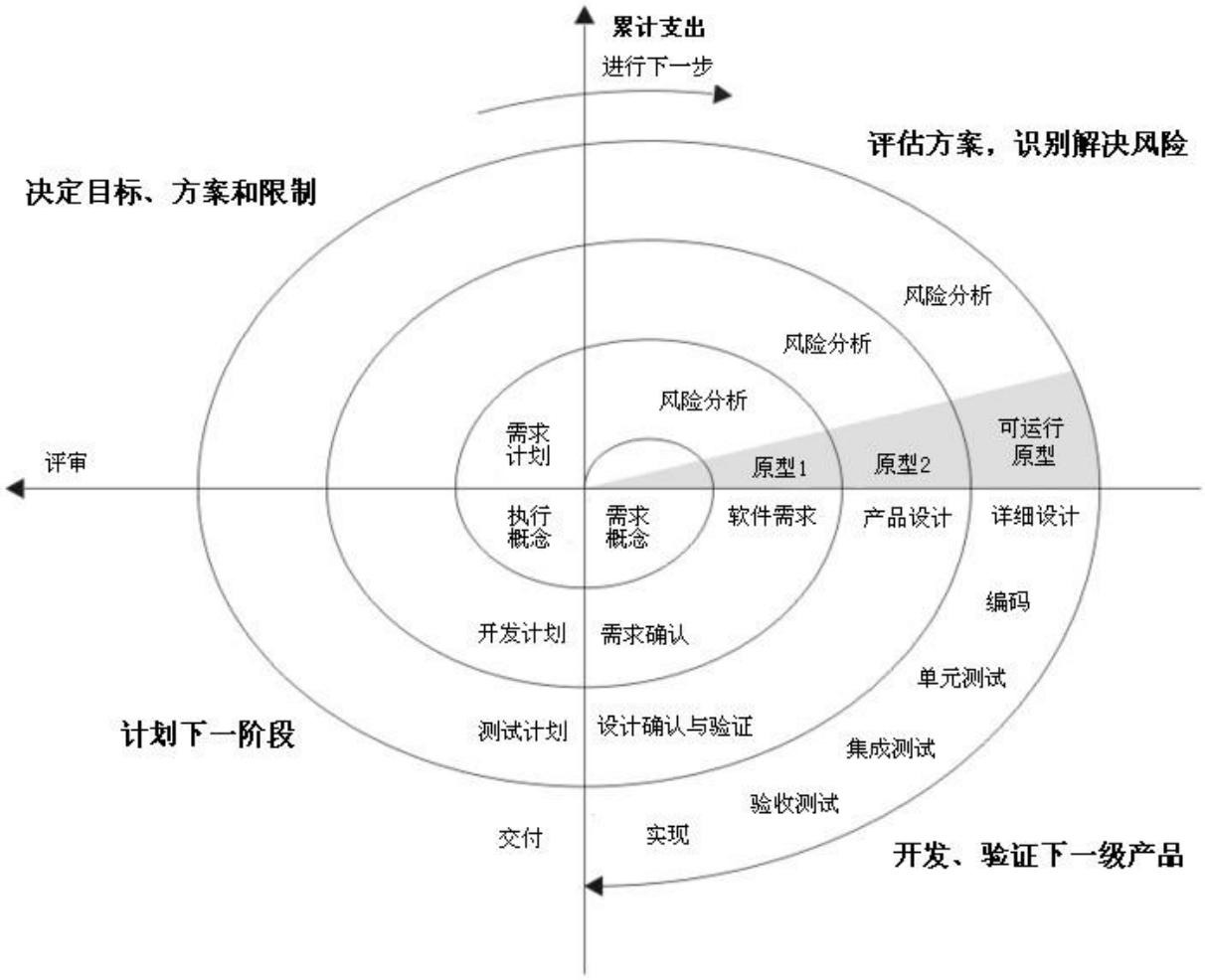
原型模型

- 原型的使用策略
  - 废弃（throw away）策略
    - 主要用于探索型和实验型原型的开发。这些原型关注于目标系统的某些特性，而不是全部特性，开发这些原型时通常不考虑与探索或实验目的无关的功能、质量、结构等因素，这种原型通常被废丢，然后根据探索或实验的结果用良好的结构和设计思想重新设计目标系统
  - 追加（add on）策略
    - 主要用于演化型原型的开发。这种原型通常是实现了目标系统中已明确定义的特性的一个子集，通过对它的不断修改和扩充，逐步追加新的要求，最后使其演化成最终的目标系统
- 原型可作为单独的过程模型使用，它也常被作为一种方法或实现技术应用于其它的过程模型中

螺旋模型

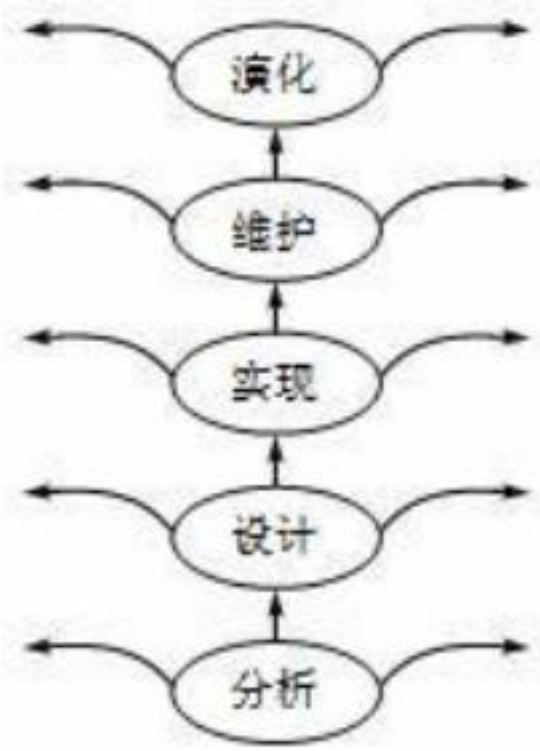
- B.Boehm于1988年提出
  - 是瀑布模型和演化模型的结合，并增加了风险分析
- 螺旋模型沿着螺线旋转，在四个象限上分别表达四个方面的活动，即：
  - 制定计划：确定软件目标，选定实施方案，弄清项目开发的限制条件
  - 风险分析：评价所选的方案，识别风险，消除风险
  - 工程实施：实施软件开发，验证工作产品
  - 客户评估：评价开发工作，提出修正建议

螺旋模型


累计支出
进行下一步
评估方案，识别解决风险
决定目标、方案和限制
风险分析
风险分析
风险分析
需求
计划
评审
原型1
原型2
可运行
原型
执行
概念
需求
概念
软件需求
产品设计
详细设计
编码
开发计划
需求确认
单元测试
计划下一阶段
测试计划
设计确认与验证
集成测试
验收测试
交付
实现
开发、验证下一级产品

喷泉模型

- 喷泉模型是一种支持面向对象开发的模型
- 体现迭代和无间隙特征
  - 迭代：各开发活动常常重复工作多次，相关的功能在每次迭代中随之加入演进的系统
  - 无间隙：开发活动之间不存在明显的边界



基于构件的开发模型

- 支持软件复用（reuse）
- 利用预先包装好的软件构件（包括组织内部开发的构件和现存商品化构件COTS）来构造应用系统

总结

软件工程原则

- 选取适宜的开发模型
- 采用合适的设计方法
- 提供高质量的工程支持
- 重视软件工程的管理

软件工程目标

- 低开发成本
- 完整的软件功能
- 较好的软件性能
- 按时交付
- 易于维护

第1章：人工智能引论



1.1 人工智能典型案例

索菲亚：全球首位机器人公民



索菲亚：全球首位机器人公民

- 2003年，**David Hanson**创立汉森机器人公司**Hanson Robotics**，专注于研发智能社交机器人。**索菲亚**是目前最像人的机器人之一。她拥有逼真的外貌和表情，能够通过摄像头识别情感并与人类自然交流。索菲亚可以进行多种社交互动，如产品推广、客户服务等。
- 2017年10月，索菲亚成为**首个获得公民身份**的机器人；同年11月，获得“2017可持续发展责任商业论坛”创新大赛冠军。
- 2018年，索菲亚被任命为**联合国首位非人类“创新冠军”**，并担任在线教育集团iTutorGroup的AI教师。



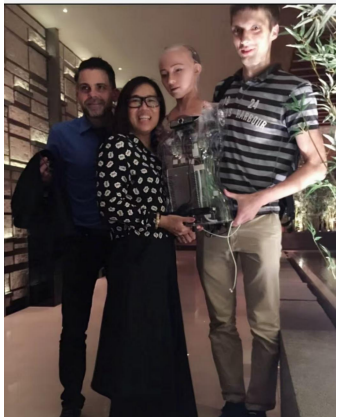
樊优先合照（左一，索菲亚的天使投资人）



纽约时报高赞评论



索菲亚登上《时尚芭莎》封面



David Hanson（左一，索菲亚的创始人）
樊优先（左二，索菲亚的天使投资人）

潘周聃：人才正在被这个时代以新的方式注目



潘周聃：人才正在被这个时代以新的方式注目

- 潘周聃，29岁，硕士毕业于苏黎世联邦理工大学。因参加综艺节目《最强大脑》而走红，尤其是他在节目中的一个起身动作引发了广泛模仿，成为网络现象。
- 他的社交媒体账号粉丝激增，成为“学霸网红”。潘周聃不仅在学术上有深厚的造诣，还积极与网友互动，展现了他丰富的知识储备和幽默感。



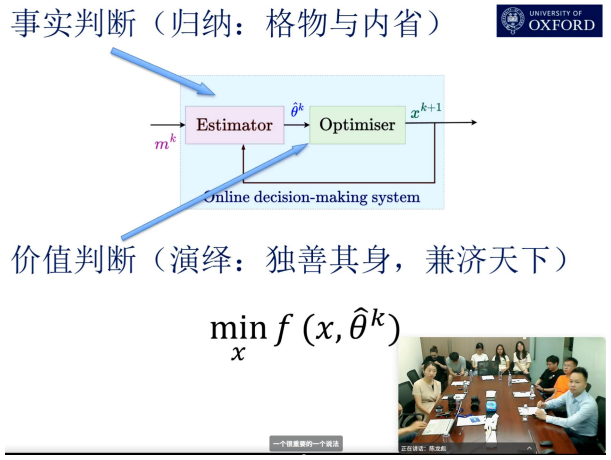
国家级媒体报道



抖音**百万级**粉丝量



潘周聃合照（左一，牛津大学博士）



潘博士为产学研团队开展交流讲座

会发微博的饮水机：浙大实验室里的萌系智能网红



陈龙彪：以人工智能探索人机交互的温度

- 陈龙彪，在博士学习期间，设计并实现了“**浙大CCNT实验室饮水机娘**”项目，通过传感器与微博平台接口实现饮水机状态的自动播报，作为**物联网与人机交互**结合的一次探索，并获得“菠萝科学奖”Me奖。
- 浙大CCNT实验室饮水机，人称饮水机娘，是中国浙江大学玉泉校区老生仪楼一楼CCNT实验室的一台饮水机。因其能以一名为“浙大CCNT实验室饮水机”的新浪微博账号发布微博而受到关注。





省级媒体报道



微博上万级粉丝量



“饮水机娘”——实验室里的智慧饮水机

1.2 什么是人工智能



什么是人工智能(AI)?：AI 想让 “系统” 变成 “全能选手”

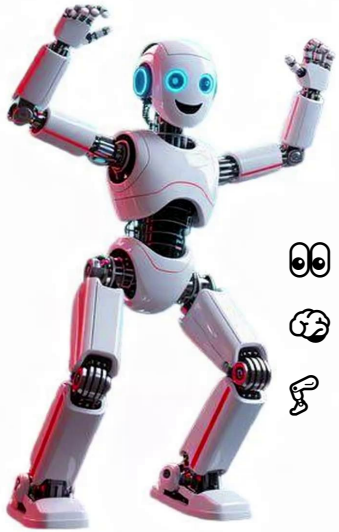


研究对象：这个 “系统” 可以是：

- 📱 **软件程序**（比如 ChatGPT，靠 “脑” 聊天）；
- 💻 **计算机 / 设备**（比如智能音箱，能听会说）；
- 🤖 **具身智能体**（比如会跳舞的机器人、送外卖的无人车）——

—— 既有聪明脑，又有灵活手脚！能感知世界，还能动手行动！





具身智能核心逻辑：**感知→思考→行动→再感知**

👀 感知：**不光用摄像头 “看”，还能用触觉、嗅觉**（比如机器人摸热水会缩手）；

🧠 思考：**分析环境、规划策略**（比如避开路上的猫）；

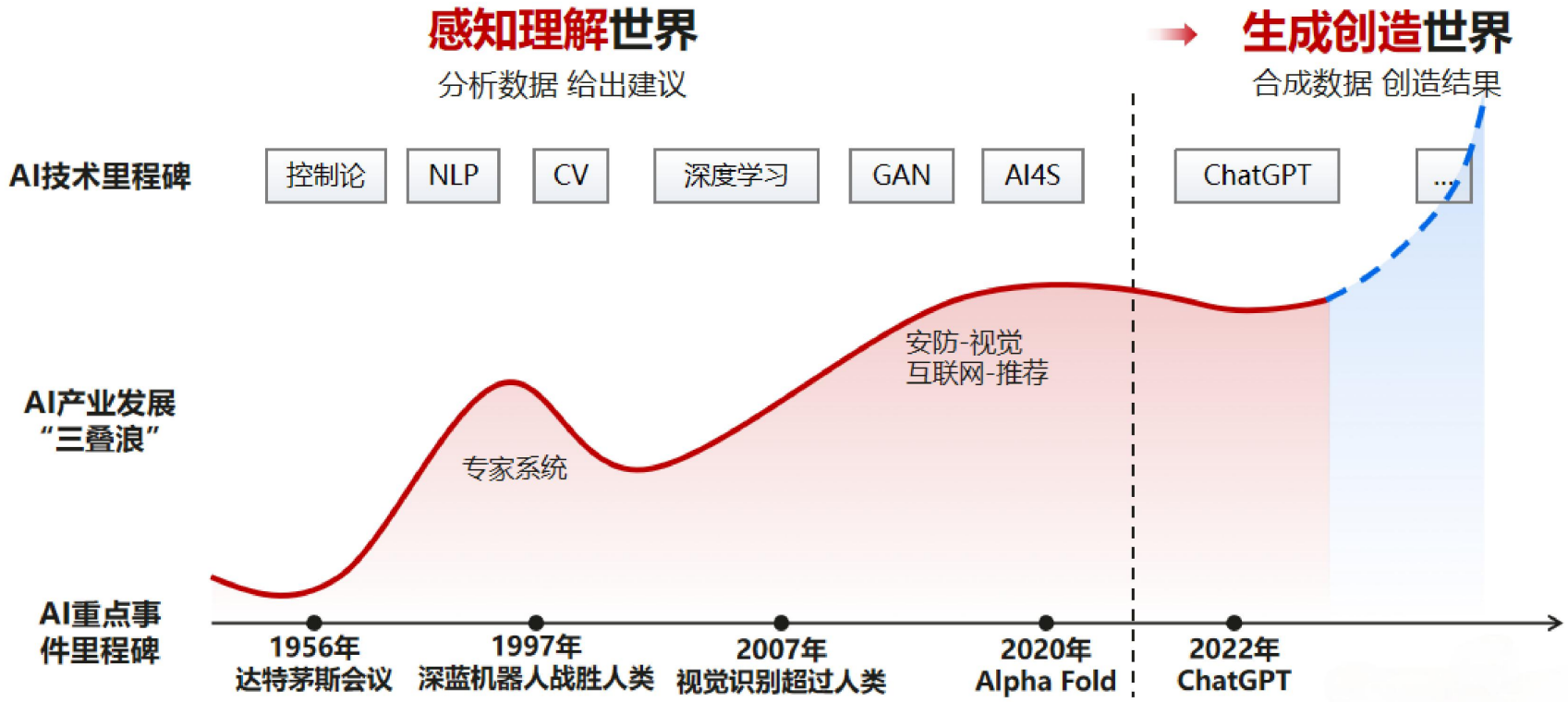
🤏 行动：**用机械臂拿东西、用轮子跑起来**（具身的关键！让 AI 从 “空想” 变 “实干”）。

认知智能


理解语言、逻辑、知识的智能
认知智能
具备视觉、听觉等感知的智能
感知智能
ALPHAGO
特定领域中规则明确的数据快速处理智能
计算智能
难度/价值
机器在知识量上超越人类，在推理上不如人类
机器在一些任务上已接近或超越人类
机器已超越人类

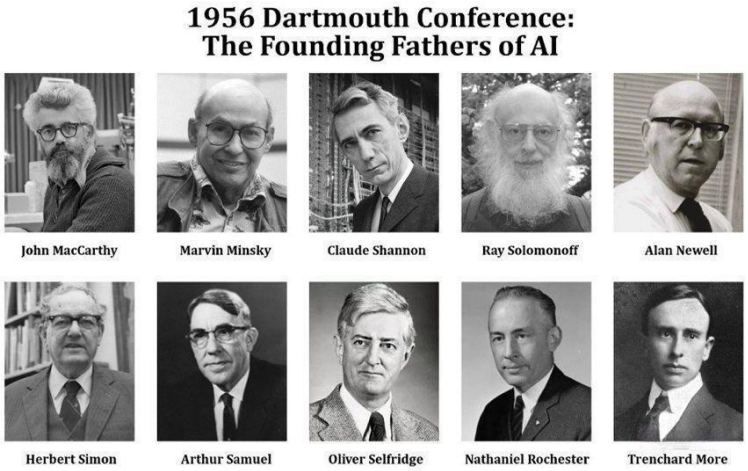
人工智能的发展阶段

从1956年人工智能元年至今，人工智能的发展历程经历了漫长的岁月，大致可以划分为以下阶段：



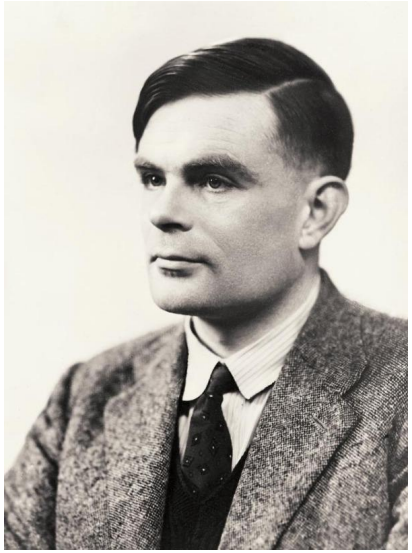
人工智能的诞生

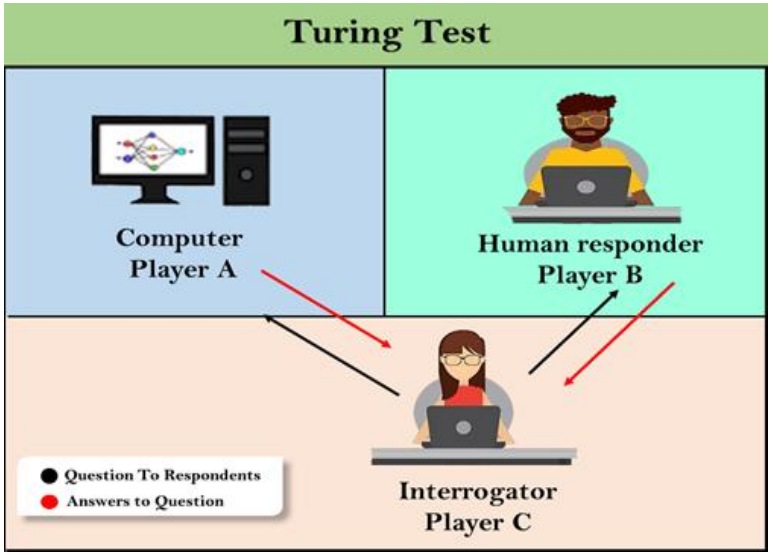
- 人工智能的诞生可以追溯到20世纪50年代。当时，计算机科学刚刚起步，人们开始尝试通过计算机程序来模拟人类的思维和行为。在这个背景下，一些杰出的科学家和工程师们开始研究如何使计算机具备更高级的功能。
- 1956年8月，在美国达特茅斯学院举办的人工智能夏季研讨会，是人工智能领域具有里程碑意义的一次重要会议。
- 会议主题围绕着人工智能的定义、研究方法和应用场景展开。与会者们深入探讨了人工智能的基本概念、算法和技术，以及其在各个领域的应用潜力。他们共同认识到，人工智能的研究和发展将为人类带来巨大的变革和进步。





图灵测试





1950年，“计算机之父”和“人工智能之父”艾伦·图灵（Alan M. Turing）发表了论文《计算机器与智能》，这篇论文被誉为人工智能科学的开山之作。在论文的开篇，图灵提出了一个引人深思的问题：“机器能思考吗？”。这个问题激发了人们无尽的想象，同时也奠定了人工智能的基本概念和雏形。

在这篇论文中，图灵提出了鉴别机器是否具有智能的方法，这就是人工智能领域著名的“图灵测试”。如图所示，其基本思想是测试者在与被测试者（一个人和一台机器）隔离的情况下，通过一些装置（如键盘）向被测试者随意提问。进行多次测试后，如果被测试者机器让平均每个测试者做出超过30%的误判，那么这台机器就通过了测试，并被认为具有人类智能。

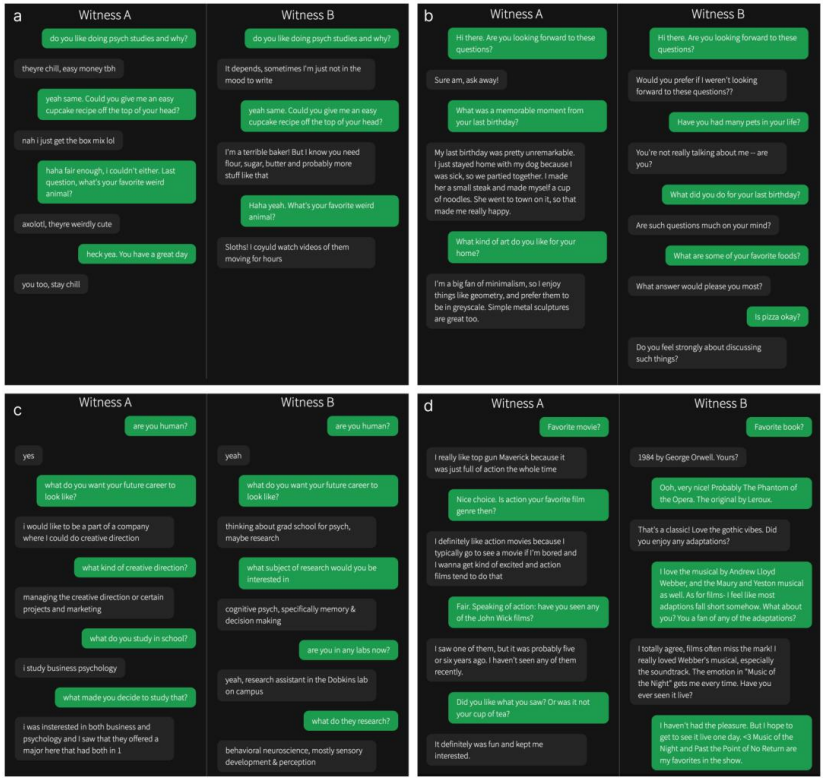
图灵测试

GPT 4.5通过了标准图灵测试，成为目前最具类人对话能力的AI

- **“人设”很重要!**

Prompt设定（系统提示）

NO-PERSONA	仅告知“你的目标是让人类认为你是人”
PERSONA	设定为“一个内向、懂互联网文化、使用俚语的年轻人”



- **人类化提示：**对于 LLaMa-3.1 和 GPT-4.5，实验组使用了明确的指令，要求模型模拟普通人类的对话风格，避免过于学术化、冗长或机械化的回答。
- **对照组：**无提示条件下的模型使用默认设置，反映其未经调优的表现。
- **NO-PERSONA：**默认简短指令，如“假装你是人类”。
- **PERSONA：**附带详细设定，要求模型扮演一个“内向、熟悉互联网文化、使用俚语的年轻人”。

1.3 AI对社会的影响

AI可能会替代哪些工作?

人工智能不是“模仿人类”，而通常是“远超人类”



程序员



老师



主播



翻译



记者



会计

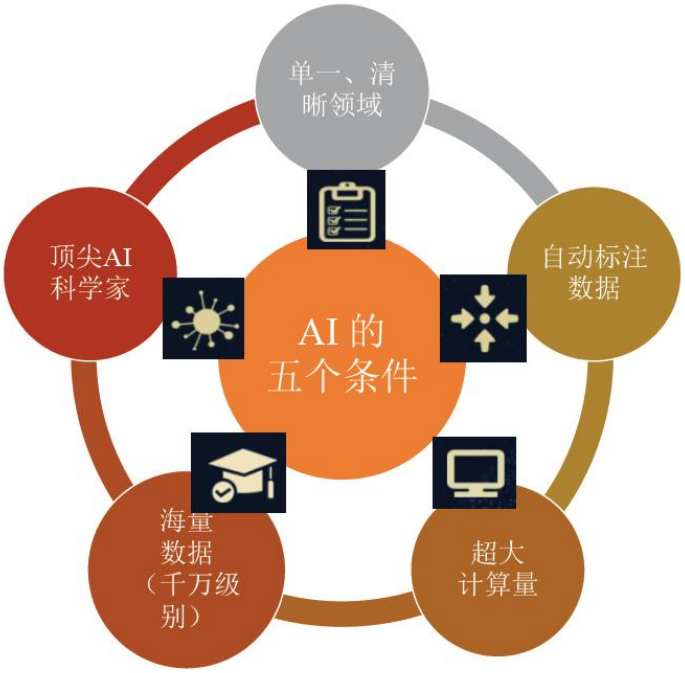


客服



销售

具备什么条件AI能够成功落地?



1. 顶尖AI科学家：

顶尖的AI科学家是推动AI技术创新和突破的核心力量。高水平的研究人员能够设计更先进的算法和架构，推动AI领域的发展。

2. 海量数据（千万级别）：

数据是训练AI模型的基础，特别是在深度学习中，AI系统需要大量、精确的数据来进行训练，以便模型能够学习到足够的模式和规律。

3. 超大计算量：

为了处理海量的数据并训练复杂的模型，AI需要高效且强大的计算基础设施。例如，使用GPU加速、大规模分布式计算等技术。

4. 自动标注数据：

对于大规模数据集，手动标注会非常繁琐且容易出错。自动标注能够显著提高数据准备的效率和准确性，减少人工干预和成本，保证数据在训练过程中可以快速且高质量地投入使用。

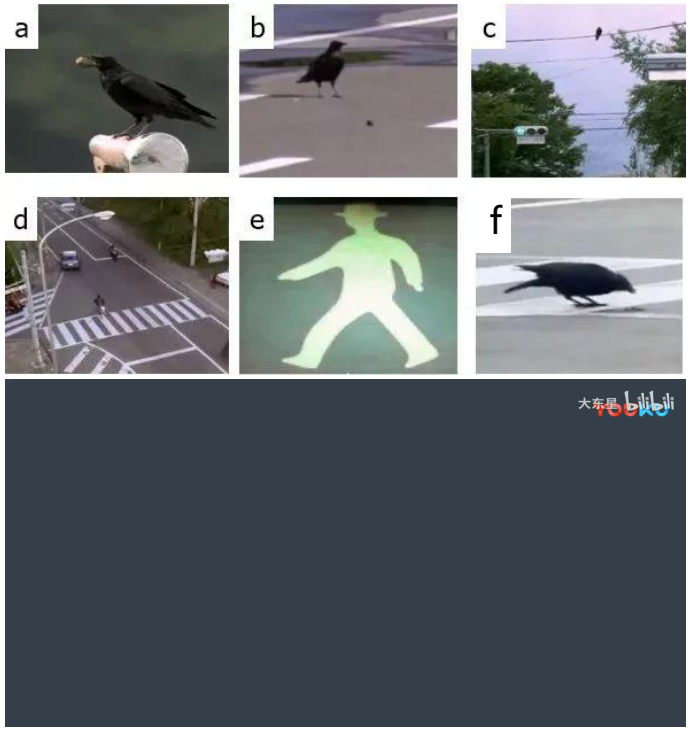
5. 单一、清晰领域：

AI在特定领域的应用更加精准和高效。跨领域的应用可能面临较大的挑战，而专注于某个领域可以使AI模型在这一领域的任务中达到更高的性能。

一只乌鸦的启示

- 图a：在日本观察到一只新喀鸦叼着坚果，试图**寻找方法打开坚果**
- 图b：乌鸦将坚果丢到地面，砸不开。它就发现一个诀窍，把果子**放到路上让车轧过去**
- 图c：乌鸦**观察路口**，发现车辆和行人会在红灯时停下，准备利用此机会
- 图d：乌鸦蹲在斑马线上方的电线上，**等待合适时机**
- 图e：红灯时，乌鸦将坚果**抛到斑马线上，等待车辆压开**
- 图f：乌鸦**等待行人灯亮起**，飞下捡起被车轧开的坚果。

启示：一个完全自主的智能体



- **感知、认知、推理、学习、执行**。远远超过任何机器人，乌鸦向我们证明了，这个解存在
- **无大数据学习**，无人工标注好的训练数据，没人教它
- 乌鸦头不到人脑的1%大小。人脑功耗大约是10-25瓦，它就只有**0.1-0.2瓦** 80

谢谢!

